

Basics of Convolutional Neural Network Operations

Deep Learning Notes

January 28, 2026

1 Introduction

This document covers the fundamental building blocks of Convolutional Neural Networks (CNNs): standard Convolution, Dilated Convolution, Max Pooling, and Average Pooling.

2 Convolution (Standard)

2.1 Concept

Convolution is the core operation of a CNN. It involves sliding a small matrix called a **kernel** (or filter) over the input image (feature map). At each position, an element-wise multiplication is performed between the kernel and the overlapping part of the input, followed by a summation.

2.2 Mathematics

For an input image I and a kernel K of size $k \times k$, the output S at position (i, j) is calculated as:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n) \quad (1)$$

where m, n iterate over the kernel dimensions.

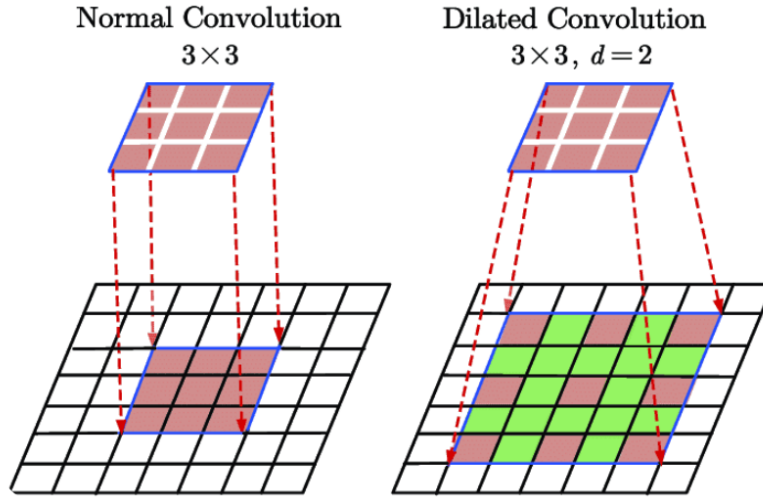
2.3 Key Parameters

- **Stride:** The step size the kernel moves at each iteration.
- **Padding:** Adding zeros around the border of the input to control the spatial dimensions of the output.

3 Dilated Convolution

3.1 Concept

Dilated convolution (also known as atrous convolution) "inflates" the kernel by inserting holes (zeros) between the kernel elements. This allows the network to have a larger



receptive field without increasing the number of parameters or the amount of computation.

3.2 Mathematics

If the dilation rate is d , the equation becomes:

$$S(i, j) = \sum_m \sum_n I(i + m \cdot d, j + n \cdot d) \cdot K(m, n) \quad (2)$$

- A dilation rate of $d = 1$ is a standard convolution.
- A dilation rate of $d = 2$ means there is one pixel gap between kernel elements.
- A dilation rate of $d = 3$ means there is two pixel gap between kernel elements.
- A dilation rate of $d = n$ means there is $n-1$ pixel gap between kernel elements.

4 Pooling Operations

Pooling is a down-sampling operation that reduces the dimensionality of the feature map, making the model more computationally efficient and reducing overfitting.

4.1 Max Pooling

Max pooling selects the **maximum** value from the window covered by the filter.

$$Y(i, j) = \max_{(m, n) \in \text{window}} \{X(i + m, j + n)\} \quad (3)$$

Use case: It preserves the most prominent features (e.g., edges, textures) and provides translation invariance.

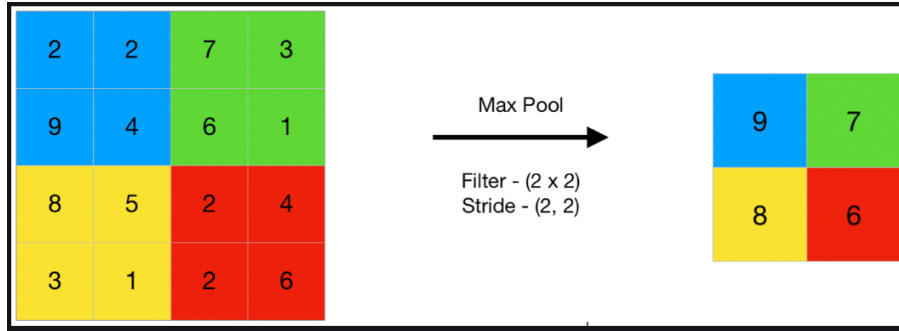


Figure 1: Max Pool



Figure 2: Average Pool

4.2 Average Pooling

Average pooling calculates the **average** value of the elements in the window.

$$Y(i, j) = \frac{1}{|W|} \sum_{(m, n) \in \text{window}} X(i + m, j + n) \quad (4)$$

Use case: It smooths the image and is often used in the final layers of a network (e.g., Global Average Pooling).

4.3 Python Setup

First, we import the necessary packages and define a simple dummy input image (1 batch, 5 height, 5 width, 1 channel) to demonstrate the operations.

Listing 1: Imports and Input Setup

```
import tensorflow as tf
import numpy as np

# Create a 5x5 image with 1 channel
# Shape: (Batch, Height, Width, Channels)
input_data = np.array([
    [1, 2, 3, 0, 1],
    [0, 1, 2, 3, 0],
    [3, 0, 1, 2, 1],
    [1, 2, 3, 0, 0],
    [0, 1, 2, 3, 0]
```

```

[0, 1, 0, 1, 2]
], dtype=np.float32).reshape(1, 5, 5, 1)

print(f"Input Shape: {input_data.shape}")

```

5 Convolution (Standard)

5.1 Concept

Convolution slides a kernel (filter) over the input. It performs element-wise multiplication and summation.

$$S(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

5.2 TensorFlow Code

Listing 2: Standard Convolution

```

# Filters=1, Kernel Size=3x3, Stride=1, No Padding
conv_layer = tf.keras.layers.Conv2D(
    filters=1,
    kernel_size=(3, 3),
    strides=(1, 1),
    padding='valid', # 'valid' means no padding
    use_bias=False
)
output_conv = conv_layer(input_data)
print(f"Conv Output Shape: {output_conv.shape}")

```

6 Dilated Convolution

6.1 Concept

Dilated convolution inserts holes (zeros) between kernel elements to increase the receptive field without increasing parameters. If dilation rate is d :

$$S(i, j) = \sum_m \sum_n I(i + m \cdot d, j + n \cdot d) \cdot K(m, n)$$

6.2 TensorFlow Code

Listing 3: Dilated Convolution

```

# Dilation Rate = 2
dilated_layer = tf.keras.layers.Conv2D(
    filters=1,
    kernel_size=(3, 3),
    dilation_rate=(2, 2), # Rate > 1 makes it dilated

```

```

padding='same',
use_bias=False
)
output_dilated = dilated_layer(input_data)
print(f"Dilated-Output-Shape: {output_dilated.shape}")

```

7 Pooling Operations

Pooling reduces dimensionality.

7.1 Max Pooling

Selects the maximum value in the window to capture the most prominent features.

$$Y(i, j) = \max_{(m, n) \in \text{window}} \{X(i + m, j + n)\}$$

Listing 4: Max Pooling

```

# Pool Size=2 X 2, Strides=2
max_pool = tf.keras.layers.MaxPooling2D(
    pool_size=(2, 2),
    strides=2
)
output_max = max_pool(input_data)
print(f"Max-Pool-Output-Shape: {output_max.shape}")

```

7.2 Average Pooling

Calculates the average value in the window to smooth the features.

$$Y(i, j) = \frac{1}{|W|} \sum_{(m, n) \in \text{window}} X(i + m, j + n)$$

Listing 5: Average Pooling

```

avg_pool = tf.keras.layers.AveragePooling2D(
    pool_size=(2, 2),
    strides=2
)
output_avg = avg_pool(input_data)
print(f"Avg-Pool-Output-Shape: {output_avg.shape}")

```